



Unidade 2

Representación e medidas da información

SUMARIO

- Tipos de datos
- Sistemas de codificación numérica
- Medidas de información
- Aritmética binaria
- Métodos para representar números enteiros
- Importancia da representación en complementos
- Métodos de enderezamento

OBXECTIVOS

- Distinguir os diferentes tipos de datos
 - Aprender a cambiar de base de numeración
 - Realizar cálculos sinxelos coas unidades de medida de uso común nos sistemas informáticos
 - Saber operar en binario
 - Coñecer os métodos para representar números de distintas maneiras
 - Coñecer as diferentes formas de enderezar datos
-

1 >> Introducción

Os ordenadores precisan información coa que traballar para poder resultar útiles. Ó estar compostos de circuitos electrónicos, atopámonos co problema de que só traballan con dous estados: pasa corrente (que representaremos cun 1) e non pasa corrente (que representaremos cun 0). Esta característica dos equipos electrónicos xera a necesidade de transformar os datos, tanto numéricos como alfanuméricos, nunha representación binaria para que o ordenador os poida procesar. Esta información divídese en ordes e datos, que poden ser de diferentes tipos atendendo ó tratamento que se lles dea.

2 >> Tipos de datos

O tratamento automático da información establece tres fases de procesamento (entrada, proceso e saída) que clasifican os datos do seguinte modo:

- **De entrada:** son os que se introducen dende soportes de información (disco duro, lapis de memoria, DVD...) ou a través dos periféricos de entrada (teclado, rato, escáner...). Correspóndense coa fase de entrada.
- **Intermedios:** son os que se obteñen e usan na fase de proceso, que ten lugar no hardware do ordenador.
- **De saída:** denomínanse tamén resultados e aparecen na fase de saída. Visualizámolos a través dos periféricos de saída (monitor, impresora...).

Os datos tamén se clasifican atendendo a se o seu valor cambia ou non durante o proceso:

- **Constantes:** permanecen inamovibles durante a execución do proceso ou programa que os usa, como por exemplo o número pi nun programa que calcula a superficie dun círculo, pois o seu valor será o mesmo para calquera dato de entrada e non variará ó longo da execución.
- **Variables:** modifícanse durante o proceso dependendo do código do programa, por exemplo os xuros que un usuario debe pagar ó banco pola súa hipoteca calculados por un programa, pois dependendo do xuro pactado (e doutras condicións) o resultado variará.

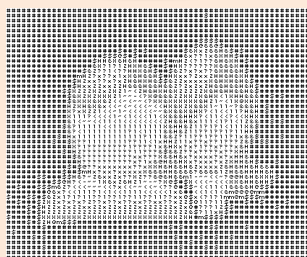
Nos ordenadores, o conxunto de instrucións execútase sobre un conxunto de datos. Esta información submínistrase mediante símbolos o caracteres:

Caracteres	
Alfabéticos	a, b, c... y, z, A, B, C... Y, Z
Numéricos	0, 1, 2, 3... 9
Especiais	· + - [] () { } ? ...
De control	Fin de liña, chío, avance de páxina...
Gráficos	♣ ♠ ® © ™ € £ ¥...

Os caracteres do primeiro e segundo grupo denomínanse **caracteres alfanuméricos** e os pertencentes ós tres primeiros grupos, **caracteres de texto**.

2.1 > Códigos de Entrada/Saída

Os sistemas de codificación alfanumérica serven para representar unha cantidade determinada de símbolos en binario. A cada símbolo corresponderalle unha combinación dun número de bits. A asignación de códigos é arbitraria, e por tanto cada fabricante podería asignar unha combinación diferente ó mesmo carácter. Para combater o caos que provocaría, créanse códigos que normalicen esta situación, e que se aceptan entre toda a comunidade informática como estándares. Os sistemas de codificación alfanumérica máis importantes son:



Imaxe en código ASCII

Co programa **ASCII-O-Matic**, dispoñible na Web, podemos transformar calquera imaxe en caracteres alfanuméricos.

- **Código BCD de intercambio normalizado (Standard BCD Interchange Code):** utiliza 6 bits, polo que pode asignar $2^6 = 64$ valores (non inclúe letras minúsculas). As veces engádeselle un bit de paridade impar, para comprobar erros, co que se ten unha lonxitude de 7 bits, pero só 64 valores válidos. O formato que seguen as palabras deste código é o seguinte:
 - **Bit de paridade (ou de verificación):** é opcional; serve para detectar erros nos caracteres.
 - **Bits de zona:** serven para distinguir entre un número e outro carácter (00 para os caracteres numéricos).
 - **Bits de posición:** para os valores numéricos, codifícase en código binario natural (agás o cero que se codifica como un 10, isto é, 1010_2).
- **EBCDIC (Extended BCD Interchange Code):** cada símbolo representámo-lo por unha combinación de 8 bits agrupados en dous bloques de catro. É o formato estendido do BCD. Neste caso asignaremos $2^8=256$ posibles combinacións de ceros e uns (codificaremos maiúsculas, minúsculas, números e incluso caracteres de control). O formato sería:
 - **Bits de zona:** se valen 00 o carácter é de control, se valen 01 trátase dun carácter especial (nin letra nin número), se valen 10 é un carácter en minúscula e se valen 11 é un carácter en maiúscula ou numérico.
 - **Bits de posición:** se os bits 2 e 3 valen 11 trátase dun carácter numérico; noutro caso trátase dunha letra.
- **ASCII (American Standard Code for Information Interchange):** este sistema utiliza unha combinación de 8 bits para representar cada símbolo, co que se poden representar un total de 256 símbolos diferentes (2^8). É o máis utilizado. Con este código podemos representar os díxitos do 0 ó 9, as letras maiúsculas do A ó Z, as minúsculas, caracteres especiais e de control.
- **Unicode:** úsase na maioría das aplicacións actuais e en Internet, así como en sistemas operativos como Windows.
- **FIELDATA:** utiliza bloques de 6 bits para representar os diferentes símbolos. O seu uso é raro e pouco estendido, xa que só se usa en ordenadores que procesan a información en bloques de 36 bits. Usábase en ordenadores Unisys.

3 >> Sistemas de codificación numérica

Un sistema de numeración é o conxunto de símbolos e regras que se utiliza para representar datos numéricos.

Caracterízanse por estar ligados a unha base que determina o número de símbolos diferentes que os compoñen. Trátase de sistemas posicionais, de forma que o valor que cada símbolo representa queda determinado pola posición que ocupa en relación ó resto de símbolos e polo factor de multiplicación correspondente.

Aqueles que nos guiamos polo sistema métrico decimal (é o habitual, aínda que os ingleses, por exemplo, usan outros sistemas de medida) estamos afeitos a usar múltiplos de 10 para representar cantidades. Isto é así porque o número de símbolos que emprega o noso sistema de codificación é exactamente 10 (do 0 ó 9).

O sistema de codificación numérica que emprega o ordenador para representar internamente as instrucións e os datos é o código binario natural, que posúe dous símbolos (0 e 1) debido ás características propias que presentan os equipos informáticos: funcionan con electricidade e é relativamente sinxelo comprobar se nun determinado instante pasa ou non a corrente eléctrica por un punto.

Cada un dos símbolos cos que se codifica cada sistema é a menor unidade de información que o sistema pode procesar e recibe o nome de **bit**.

Os ordenadores, a nivel interno, tamén utilizan códigos agrupados (octal e hexadecimal), debido a que fai anos facilitaban a labor dos programadores porque programar en binario (ensamblador ou código máquina) é moi irritante e provoca erros imprevisibles se secuenciamos mal os uns e os ceros.

É posible agrupar os bits deste xeito porque é moi doado converter un número en base 2 ó correspondente noutra base maior sempre que sexa potencia de dous (pois existe unha correspondencia directa entre elas), pero non a outras como base 5 ou base 7.

Polo tanto, os sistemas de numeración que o ordenador emprega para representar a información que estamos afeitos a usar en decimal son:

- **Binario:** é un sistema en base 2 (dous símbolos: 0 e 1). É o que se deriva directamente do paso da corrente eléctrica polos compoñentes electrónicos do equipo, polo que o ordenador o usa a nivel interno. Cada símbolo decimal represéntase mediante unha combinación de catro bits, como podemos ver na táboa lateral.
- **Octal:** é un sistema en base 8 (oito símbolos: do 0 ó 7), polo que os números se representan posicionalmente por potencias de 8. Cada símbolo en base 8 equivale a 3 bits no sistema binario.
- **Hexadecimal:** é un sistema en base 16 (dezaseis símbolos: do 0 ó 9 e do A ó F). Os símbolos alfabéticos do A ó F representan os números do 10 ó 15 do sistema decimal, polo que a súa representación decimal precisa dous símbolos. Cada símbolo en base 16 equivale a 4 bits no sistema binario.

3.1 > Conversión dun sistema de numeración a outro

Calquera base a decimal

Para pasar de calquera base a decimal simplemente aplicamos o teorema fundamental da numeración que aparece na marxe da páxina, substituíndo B pola base correspondente.

Así, se queremos pasar un número de binario a decimal teremos que B é 2, se desexamos pasar de octal a decimal B será 8 e se queremos pasar de hexadecimal a decimal tomaremos como base 16.

Cada unha delas posúe pequenas diferenzas respecto ás demais, pero basicamente en todas aplicamos un sumatorio sobre unha serie de potencias.

Con sistemas de numeración que utilicen máis símbolos dos que ten o código decimal, teremos que utilizar unha tradución deses símbolos antes de aplicar o teorema.

Sistemas de numeración			
Decimal	Binario	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

... 100000 10000 1000 100 10 1
 ... 10⁵ 10⁴ 10³ 10² 10¹ 10⁰
...654321

Teorema fundamental de la numeración (TFN)

Tódolos sistemas posicionais están baseados neste teorema, que relaciona cantidades de calquera sistema de numeración con esas mesmas cantidades no sistema decimal.

Ven dado pola fórmula seguinte:

$$\sum X_i * B^i$$

Onde:

X é o valor absoluto do dígito en cuestión.

i é a posición que ocupa o dígito respecto ó punto decimal.

B é a base.

Así:

$$80,5 = 8 \cdot 10^1 + 0 \cdot 10^0 + 5 \cdot 10^{-1}$$

Exemplos

Binario a decimal

Obter o valor decimal do número 1011101.01₂, sendo ₂ a base.

Na parte enteira, determinamos as posicións de dereita a esquerda a partir do punto decimal:

$$1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

Para a parte decimal traballamos de esquerda a dereita:

$$0 \cdot 2^{-1} + 1 \cdot 2^{-2}$$

O número buscado será a suma de ambas partes:

$$1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} = 93.25_{10}$$

Octal a decimal

Queremos obter o valor decimal do número 117.45₈.



Mensaxe diabólica

Bender deixa unha clara mensaxe nun dos capítulos de Futurama. A quen representa o número que escribe no espello?



Sistema hexadecimal

Este sistema emprega letras para representar parte dos seus símbolos, polo que deberemos traducir, ben mentalmente ou ben por medio da escritura, esas letras a valores numéricos antes de efectuar as operacións matemáticas.



Díxitos infinitos

Un número cun só díxito na súa parte fraccionaria en base 10 pode xerar un número con infinitos díxitos fraccionarios en base 2. Comprobámolo co número $0.2_{(10)}$.

$$\begin{aligned} 0.2 \cdot 2 &= 0.4 \\ 0.4 \cdot 2 &= 0.8 \\ 0.8 \cdot 2 &= 1.6 \\ 0.6 \cdot 2 &= 1.2 \\ 0.2 \cdot 2 &= 0.4 \\ &\dots \end{aligned}$$

Unha das operacións repítese, polo que será un número periódico: $0.00110011\dots_2$.

Parte enteira

$$1 \cdot 8^2 + 1 \cdot 8^1 + 7 \cdot 8^0$$

Parte decimal

$$4 \cdot 8^{-1} + 5 \cdot 8^{-2}$$

O número buscado será a suma de ambas partes:

$$1 \cdot 8^2 + 1 \cdot 8^1 + 7 \cdot 8^0 + 4 \cdot 8^{-1} + 5 \cdot 8^{-2} = 79.578125_{(10)}$$

Hexadecimal a decimal

Queremos obter o valor decimal do número $A03F.07_{(16)}$.

Parte enteira

$$A \cdot 16^3 + 0 \cdot 16^2 + 3 \cdot 16^1 + F \cdot 16^0$$

Parte decimal

$$0 \cdot 16^{-1} + 7 \cdot 16^{-2}$$

Temos que ter en conta que hai que substituír as letras polo valor equivalente en decimal:

Parte enteira

$$10 \cdot 16^3 + 0 \cdot 16^2 + 3 \cdot 16^1 + 15 \cdot 16^0$$

Parte decimal

$$0 \cdot 16^{-1} + 7 \cdot 16^{-2}$$

O número buscado será a suma de ambas partes:

$$10 \cdot 16^3 + 0 \cdot 16^2 + 3 \cdot 16^1 + 15 \cdot 16^0 + 0 \cdot 16^{-1} + 7 \cdot 16^{-2} = 41\,023.02734375_{(10)}$$

Decimal a binario

Para realizar as conversións inversas, empréganse algoritmos diferentes. En primeiro lugar temos que diferenciar entre a parte enteira e a parte decimal.

Para calcular a parte enteira dividimos o número orixinal entre a base e repetimos o procedemento para os cocientes que vamos obtendo. Os restos destas divisións e o último cociente son as cifras buscadas, que sempre serán 0 ou 1. O último cociente é o díxito máis significativo e o menos significativo é o primeiro resto.

Para calcular a parte decimal multiplicamos a parte decimal do número orixinal pola base e repetimos o procedemento sucesivamente coas partes decimais dos números obtidos. A secuencia de partes enteiras que obtemos é a representación en base 2 da parte decimal do número. Este procedemento termina cando xa non hai parte decimal.

Isto xera un problema, pois se almacenamos un número binario nun ordenador cun número prefixado de bits (polo tanto, finito), teremos que recortar o número de cifras co que o representemos co fin de gardalo nese tamaño finito.

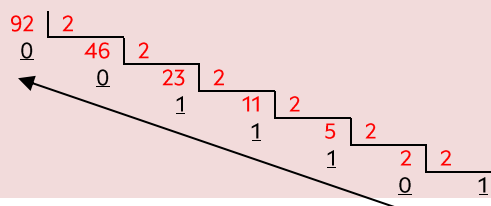
Podemos comprobar doadamente que un número decimal con díxitos fraccionarios pode dar lugar a un número binario con infinitos díxitos, tal e como sucede ó tentar calcular o valor binario do número $0.2_{(10)}$: partindo dun só díxito decimal temos un número en binario periódico puro que, á forza, terá un erro por truncamento.

Exemplos

Decimal a binario

Obter o valor binario do número $92.375_{(10)}$.

Primeiro actuamos sobre a parte enteira do número e dividímolos sucesivamente entre a base na que queremos representar o número inicial para, finalmente, ter en conta os díxitos dos restos e o último cociente:



De modo que a parte enteira sería 1011100_2 .

Agora, traballamos coa parte decimal e multiplicamos sucesivamente pola base para obter a tradución, sempre desprezando a parte enteira, ata atopar unha operación repetida (número periódico en binario) ou unha multiplicación por cero (número exacto en binario):

$$\begin{array}{rcl} 0.375 \cdot 2 & = & 0.75 \\ 0.75 \cdot 2 & = & 1.5 \\ 0.5 \cdot 2 & = & 1.0 \\ 0.0 \cdot 2 & = & 0.0 \end{array}$$

Polo tanto a parte decimal será 0110 e o número buscado 1011100.011_2 .

Decimal a outras bases

O procedemento é similar ó paso de decimal a binario, pero dividindo e multiplicando agora pola base en cuestión ($B = 8$ se estamos a traballar con codificación octal e $B = 16$ se o facemos con codificación hexadecimal).

Binario a octal

Se temos un número en binario e queremos transformalo no seu equivalente en octal, dividímo-lo en ternas de tres valores a partir do punto fraccionario; cada terna equivale a un símbolo en octal.

Se o número de díxitos non é un múltiplo de 3, os díxitos da terna incompleta complétanse con ceros, aplicándoos de forma que non modifiquen o número: pola esquerda, se é na parte enteira, ou pola dereita, se é na parte fraccionaria.

Binario a hexadecimal

Agora, en vez de separar o número binario en ternas, separamos os díxitos en grupos de catro elementos (cuaternas) tomando de novo como punto de referencia ó punto decimal; cada cuaterna equivale a un símbolo en hexadecimal.

Se o número de díxitos non é múltiplo de 4, os díxitos da cuaterna incompleta complétanse con ceros, pola esquerda para a parte enteira e pola dereita para a parte decimal.

Exemplos

Binario a octal

Obter o valor octal do número 1111000.11001_2 .

Dividimos o número en ternas, completando con ceros aquelas que o precisen:

001 111 000 . 110 010

Cada terna equivale a un símbolo en base octal (a súa interpretación en binario natural):

1 7 0 . 6 2

Polo tanto o número final obtido é o 170.62_8 .

Binario a hexadecimal

Obter o valor hexadecimal do número 11111100000.1111110101_2 .

Dividimos o número en cuaternas, completando con ceros aquelas que o precisen:

1111 1110 0000 . 1111 1110 1010

Cada unha correspóndese cun símbolo en base hexadecimal (a súa interpretación en binario natural):

F E 0 . F E A

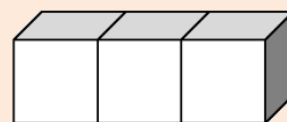
Así, o número final obtido é o $FE0.FEA_{16}$.

$123.456 \Rightarrow 123.456 \Rightarrow 123$



Erro por truncamento

O erro que se comete ó desprezar unha serie de decimais denomínase **erro por truncamento**. Ó non usar o código decimal para almacenar os datos (por motivos de eficiencia) estaremos cometendo este tipo de erro. É o mesmo problema que presentan as calculadoras (as primeiras só permitían 8 ou 10 díxitos) ou as follas de cálculo (a partir do 15º decimal) e obríganos a traballar con números aproximados en vez de reais.



Bases non potencia de dous

Se queremos pasar dunha base que non é potencia de dous a binario (ou viceversa) teremos que facer sempre un paso intermedio: **pasar a base decimal**.

Bases potencia de dous a binario

Neste caso substituímos cada símbolo (octal ou hexadecimal) polos díxitos binarios que lle correspondan (tres ou catro). Deste xeito simplificamos o noso código á hora de programar, pois en vez de ter que escribir 3 ou 4 uns e ceros (dependendo de se se trata de código octal ou hexadecimal) escribimos soamente o símbolo correspondente na base agrupada. Isto permite, ademais de obter menos erros, unha tradución máis rápida dos números binarios ós decimais e viceversa.

Exemplos

Octal a binario

Obter o valor binario do número $521.17_{(8)}$. A cada díxito octal correspóndelle unha terna en binario natural:

Dividimos o número en ternas, completando con ceros aquelas que o precisen:

5	2	1	.	1	7
101	010	001	.	001	111

Polo tanto, o número final obtido é o $101010001.001111_{(2)}$.

Hexadecimal a binario

Obter o valor binario do número $ACDC.BAD_{(16)}$.

A cada símbolo hexadecimal correspóndelle un bloque de catro díxitos en binario natural:

A	C	D	C	.	B	A	D
1010	1100	1101	1100	.	1011	1010	1101

De modo que o número final obtido é o $1010110011011100.101110101101_{(2)}$.

Hexadecimal a octal e viceversa

Para converter un número hexadecimal a octal, o procedemento máis sinxelo é pasar de hexadecimal a binario e de binario a octal, aproveitándonos de que se trata de códigos agrupados, pois todas elas son potencias de dous.

O único que temos que facer é cambiar os símbolos hexadecimais polo seu equivalente binario e volvelos agrupar, esta vez de tres en tres, para obter os símbolos equivalentes en código octal.

Para levar a cabo o caso contrario, pasaríamos de octal a binario e de este a hexadecimal.

Exemplos

Octal a hexadecimal

Obter o valor hexadecimal do número $0.07_{(8)}$.

A cada símbolo octal correspóndelle un bloque de tres díxitos en binario natural:

0	.	0	7
000	.	000	111

Agrupámoslos agora de catro en catro e calculamos a súa equivalencia en hexadecimal:

0000	.	0001	1100
0	.	1	C

Polo tanto o número final obtido é o $0.1C_{(16)}$.

4 >> Medidas de información

No sistema binario so existen dous símbolos diferentes, 0 e 1. Unha información que só pode tomar como valores o 0 e o 1 denomínase **bit** e forma a unidade básica de información.

Un ordenador, debido á súa construción baseada fundamentalmente en circuitos electrónicos dixitais, traballa co sistema binario (dous símbolos, o 0 e o 1). Este é o motivo que nos obriga a transformar internamente tódolos nosos datos, tanto numéricos como alfanuméricos, a unha representación binaria para que a máquina sexa capaz de procesalos.

Polo xeral, cando un fabricante fala de 4 GB de memoria RAM, fala de 4 GiB... cando di que a caché é de 256 KB, quere dicir que ten 256 KiB... cando di que un disco duro ten 500 GB, quere dicir que ten 500 GB (500 · 1 000 MB)... habitualmente, as capacidades das memorias principais usan múltiplos de 1 024 e as capacidades das memorias secundarias usan múltiplos de 1 000.

O problema é que ata o ano 1999 non existían os KiB, MiB, GiB... e tampouco se utilizaban as potencias de dez cando se medía a información.

Antes:

1KB = 1 024 bytes.

1MB = 1 024 KB bytes.

...

Isto hoxe en día non debería ser correcto. As táboas do lateral amosan como se teñen que nomear e facer as conversións segundo os estándares.

Pero... moitos fabricantes e comerciantes utilizan aínda a nomenclatura incorrecta.

4.1 > Engano na capacidade dos discos

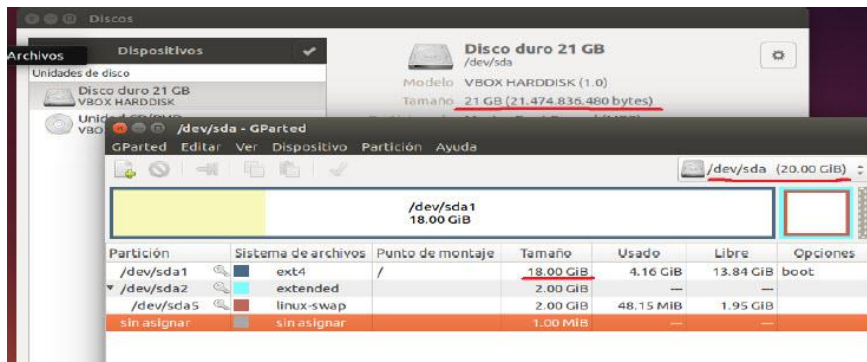
Supoñamos que queremos instalar un disco duro de 20 GB, como é unha memoria secundaria usamos a potencia de 10, polo que a capacidade sería:

$$20 \cdot 1\,000 \cdot 1\,000 \cdot 1\,000 = 2 \cdot 10^{10} \text{ bytes}$$

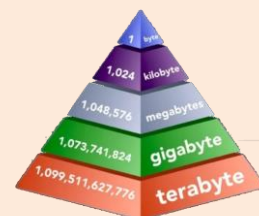
Se o instalamos nunha máquina que ten como sistema operativo Linux esta será a capacidade que lle asigne a ese disco o sistema operativo pero se ten Windows teremos que o recoñece cunha maior capacidade:

$$20 \cdot 1\,024 \cdot 1\,024 \cdot 1\,024 = 21\,474\,836\,480 \text{ bytes}$$

E se instalamos un Ubuntu nunha máquina en VirtualBox baixo o sistema operativo Windows cun disco de 20 GB... pois medra.



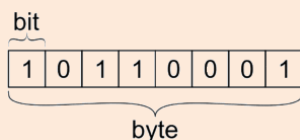
Windows utiliza un sistema binario para os GB e non utiliza a medida GiB... e Linux utiliza un sistema binario para os GiB e un sistema decimal para os GB. Pero isto lévanos a engano cando a publicidade aproveita o cambio de sistema de capacidade.



Medidas de capacidade

Byte: Agrupación de 8 bits			
abreviatura	unidade	equivalencia	potencia
KB	kilobyte	1 000 bytes	10^3
MB	megabyte	1 000 KB	10^6
GB	gigabyte	1 000 MB	10^9
TB	terabyte	1 000 GB	10^{12}
PB	petabyte	1 000 TB	10^{15}
EB	exabyte	1 000 PB	10^{18}
ZB	zettabyte	1 000 EB	10^{21}
YB	yottabyte	1 000 ZB	10^{24}

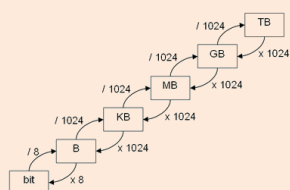
Byte: Agrupación de 8 bits			
abreviatura	unidade	equivalencia	potencia
KiB	kibibyte	1 024 bytes	2^{10}
MiB	mebibyte	1 024 KiB	2^{20}
GiB	gibibyte	1 024 MiB	2^{30}
TiB	tebibyte	1 024 GiB	2^{40}
PiB	pebibyte	1 024 TiB	2^{50}
EiB	exbibyte	1 024 PiB	2^{60}
ZiB	zebibyte	1 024 EiB	2^{70}
YiB	yobibyte	1 024 ZiB	2^{80}



bit ou byte

Non se debe confundir unha medida coa outra, un byte son 8 bits.

Recorda, **kB** é o mesmo que **KB** pero non que **kb** ou **Kb**.



Como pasamos dunha medida a outra?

É un procedemento sinxelo, por exemplo, para pasar 24 756 bits a KiB e KB faremos as seguintes operacións:

$$24\,756 / 8 / 1\,000 = 3.072 \text{ KB}$$

$$24\,756 / 8 / 1\,024 = 3 \text{ KiB}$$

Se queremos é pasar 6 291 456 bytes a MiB e MB teremos:

$$6\,291\,456 / 1\,024 / 1\,024 = 6 \text{ MiB}$$

$$6\,291\,456 / 1\,000 / 1\,000 = 6.29 \text{ MB}$$



Ancho de banda

Mídese en información/segundo, polo tanto:

$$1 \text{ bps} = 1 \text{ bit/s}$$

$$1 \text{ byte/s} = 1 \text{ B/s} = 8 \text{ bits/s}$$

$$1 \text{ KB/s} = 1\,000 \text{ B/s}$$

$$1 \text{ MB/s} = 1\,000 \text{ KB/s} = 1\,000\,000 \text{ B/s}$$

Cal é a capacidade real dun disco duro SATA anunciado de 500 GB?

Os fabricantes fan a trampa de pasar dunha medida a outra realizando multiplicacións/divisións por/entre 1 000 en vez de 1 024:

$$500 \text{ GB} \cdot 1\,000 \cdot 1\,000 \cdot 1\,000 = 500\,000\,000\,000 \text{ bytes}$$

Pero en realidade o sistema operativo Windows recoñece como:

$$500\,000\,000\,000 / 1\,024 / 1\,024 / 1\,024 = 465.66 \text{ GB}$$

ou máis formalmente **465.66 GiB**

Débese ter en conta:

- Segundo o fabricante a cantidade do disco pode variar lixeiramente e para aforrar costes sóse redondear á baixa
- A capacidade expresada con prefixo decimal resulta nunha cifra maior que si se expresara con prefixo binario
- Canta maior capacidade ten un disco duro, maior é a discrepancia entre as cifras que expresan esta capacidade con prefixo decimal e binario

4.2 > Frecuencia dun bus e Taxa de transferencia

Outra característica importante do procesador é a frecuencia á que funciona, mídese en Hz (Hertz ou ciclos/segundo) e é a velocidade á que executa cada instrución.

Aplícanse múltiplos de 1 000 en 1 000:

$$1 \text{ Hz} = 1 \text{ ciclo/s}$$

$$1 \text{ KHz} = 1\,000 \text{ Hz}$$

$$1 \text{ MHz} = 1\,000 \text{ KHz}$$

$$1 \text{ GHz} = 1\,000 \text{ MHz}$$

Cómpre destacar que non se pode comparar directamente a frecuencia de dous procesadores con microarquitectura distinta.

Aínda que un vaia máis rápido có outro (a maior frecuencia) non significa que logre executar máis microinstrucións... pode ser que a FPU (Unidade de Punto Flotante) dun procesador a 2.2 GHz sexa moito máis rápida que a doutro procesador a 2.5 GHz.

Isto é debido a que existen buses que en cada ciclo fan varias transferencias (unha, dúas, tres...) polo que na actualidade os fabricantes poden utilizar outra unidade para indicar a velocidade dun bus: as T/s ou Transferencias/segundo:

$$1 \text{ MT/s} = 10^6 \text{ transferencias/segundo}$$

$$1 \text{ GT/s} = 10^9 \text{ transferencias/segundo}$$

Por exemplo, se un bus traballa a 100 MHz e fai 4 transferencias en cada ciclo, teremos:

$$100 \text{ Mciclos/s} \cdot 4 \text{ transferencias en cada ciclo} = 400 \text{ MT/s}$$

Para calcular o ancho de banda (taxa de transferencia) sempre teñen que indicar o ancho do bus.

É a velocidade á que se transmiten os datos por unha canle, tamén se pode denominar bit rate

$$\text{Taxa de transferencia} = \text{Ancho bus} \cdot \text{Velocidade do bus (frecuencia)}$$

Por exemplo, se teño un bus que ten un ancho de 32 bits, e traballa a 200 Hz e fai unha transferencia por ciclo, quere dicir que fai 200 transferencias de datos por segundo, polo que para calcular o ancho de banda teremos:

$$200 \text{ ciclos/s} \cdot 32 \text{ b/transferencia} \cdot 1 \text{ transferencia/ciclo} \cdot 1 \text{ B/8 b} = 800 \text{ B/s}$$

4.3 > Velocidade de procesamento

A velocidade de procesamento mídese en Hz e indica o número de ciclos partido por segundo ó que traballa o microprocesador. Cada instrución do xogo de instrucións que entende o procesador leva unha serie de ciclos a súa execución... haberá instrucións que con dous ciclos xa se executan, outras tres, outras catro...

$$1 \text{ Hz} = 1 \text{ ciclo/s}$$

$$1 \text{ KHz} = 1\,000 \text{ Hz}$$

$$1 \text{ MHz} = 1\,000 \text{ KHz}$$

Por exemplo, se un microprocesador ten unha frecuencia de 300 KHz e un programa ten 100 instrucións das cales o 25% se executan nun só ciclo, un 50% necesitan dous ciclos para executarse e o resto necesitan 5 ciclos para a súa execución, cantos ciclos de microprocesador necesita o programa para executarse enteiro?

Temos:

$$25 \text{ instrucións} \cdot 1 \text{ ciclo/instrución}$$

$$50 \text{ instrucións} \cdot 2 \text{ ciclos/instrución}$$

$$25 \text{ instrucións} \cdot 5 \text{ ciclos/instrución}$$

o que equivale a 250 ciclos

$$300 \text{ KHz} = 300 \cdot 1\,000 \text{ ciclos/s} \Rightarrow 1 / 300\,000 \text{ s/ciclo}$$

$$250 \text{ ciclos} \cdot 1 / 300\,000 \text{ s/ciclo} = 0.00083333 \text{ s}$$

5 >> Aritmética binaria

Entendemos por aritmética binaria as operacións aritméticas e lóxicas feitas con variables binarias. As operacións aritméticas básicas son as seguintes:

Adición

O símbolo do operador da adición é o símbolo máis +, os elementos chámanse sumandos e o resultado suma.

A suma de dous díxitos 1 binarios dá como resultado dous bits, un de suma (0) e un de arrastre (1).

Cando sumamos fraccións de números binarios ocorre o mesmo que cando sumamos en decimal, polo tanto o punto fraccionario non inflúe no procedemento.

Exemplos

Adición

Sumar, en binario, os números decimais 54.25 e 26.8125.

Sumamos en paralelo en decimal e binario. Cada número decimal ten o seu correspondente binario: ó 54.25 correspóndelle o 110110.01₂ e ó 26.8125 o 11010.1101₂.

1 1	Díxitos de arrastre	1 1 1 1	1 1
5 4 . 2 5 0 0		1 1 0 1 1 0	. 0 1 0 0
+ 2 6 . 8 1 2 5		+ 1 1 0 1 0	. 1 1 0 1
8 1 . 0 6 2 5		1 0 1 0 0 0 1	. 0 0 0 1

Polo tanto o número final obtido é o 1010001.0001₂.

Subtracción



Rendemento dunha computadora

Para medir o rendemento dunha computadora, especialmente en cálculos científicos que usan gran número de operacións en coma flotante, úsase a medida **FLOPS** (Float Point Operation per Second) que son as operacións en punto flotante por segundo que é quen de facer.

$$1 \text{ MFLOPS} = 10^6 \text{ FLOPS}$$

$$1 \text{ GFLOPS} = 10^9 \text{ FLOPS}$$

$$1 \text{ TFLOPS} = 10^{12} \text{ FLOPS}$$

...

OLLO!!! Non se debe tomar como única medida para valorar a capacidade dun ordenador.



Aritmética binaria

A	B	+	-	·	/
0	0	0	0	0	***
0	1	1	1**	0	0
1	0	1	1	0	∞
1	1	0*	0	1	1

* e levamos un 1 á posición inmediata superior

** e levamos un -1 á posición inmediata superior

*** indeterminado

O símbolo do operador da subtracción é o símbolo -, os elementos chámanse minuendo e subtraendo e o resultado diferenza.

Do mesmo xeito, as regras para a resta en binario son iguais ás da resta en decimal; polo tanto, se o subtraendo é maior que o minuendo (o que, con números binarios, só ocorre cando o subtraendo é 1 e o minuendo é 0), o resultado da resta é 1 e débese levar un -1 á seguinte posición de maior valor.

Ó restar fraccións binarias, o punto (ou coma) decimal non afecta ó procedemento.

Cando o minuendo é negativo, a resta convértese nunha suma, aínda que se mantén o signo menos que indica un número negativo.

Exemplos

Subtracción

En binario, restarlle ó número decimal 54.25 o número 26.8125.

Restamos en paralelo en decimal e binario.

-1 -1	-1 -1	-1 -1	-1 -1 -1	-1 -1 -1
5 4 . 2 5 0 0	1 1 0 1 1 0 . 0 1 0 0			
- 2 6 . 8 1 2 5	- 1 1 0 1 0 . 1 1 0 1			
2 7 . 4 3 7 5	0 1 1 0 1 1 . 0 1 1 1			

Polo tanto o número final obtido é o 11011.0111₂.

Multiplicación

O símbolo do operador da multiplicación é o punto · (aínda que as veces tamén toparemos o asterisco ou a aspa), os elementos chámanse multiplicando e multiplicador e o resultado produto.

As regras para a multiplicación binaria son tamén as mesmas que para a multiplicación decimal.

O punto colócase exactamente igual que ó traballar con números decimais.

No que respecta ós signos, aplícanse as mesmas regras que no sistema decimal: se os signos do multiplicando e do multiplicador son iguais, o signo do produto é positivo e, se son diferentes, o signo do produto é negativo.

Exemplos

Multiplicación

Multiplicar, en binario, os números decimais 54.25 e o número 5.5.

Multiplicamos en paralelo en decimal e binario.

Cada número decimal ten o seu correspondente binario: ó 54.25 correspóndelle o 110110.01₂ e ó 5.5 o 11010.1101₂.

5 4 . 2 5	1 1 0 1 1 0 . 0 1
· 5 . 5	· 1 0 1 . 1
2 7 1 2 5	1 1 0 1 1 0 0 1
2 7 1 2 5	1 1 0 1 1 0 0 1
2 9 8 . 3 7 5	0 0 0 0 0 0 0 0
	1 1 0 1 1 0 0 1
	1 0 0 1 0 1 0 1 0 . 0 1 1

Polo tanto, o número final obtido é o 100101010.011₂.

División

O símbolo do operador da división é a barra $/$, os elementos chámanse dividendo e divisor e o resultado cociente.

As regras para a división binaria son as mesmas que para a división decimal; polo tanto, a división por cero non é posible, xa que o resultado sería unha forma indeterminada ou infinito, valores que un ordenador non pode procesar.

No que respecta á coma (ou punto) que separa a parte enteira da fraccionaria, as operacións realízanse igual que cos números decimais: primeiro, cómpre eliminar o punto do divisor desprazando o punto do dividendo cara á dereita tantas posicións como cifras decimais teña o divisor (ou engadindo á dereita o número correspondente de ceros).

Exemplos

División

Dividir, en binario, o número decimal 9 entre 3.

Realizamos a división en paralelo en decimal e binario. Cada número decimal ten un equivalente en binario: o 9 correspóndese con $1001_{(2)}$ e o 3 con $11_{(2)}$.

$$\begin{array}{r}
 9 \overline{) 3} \\
 \underline{9} \\
 0
 \end{array}
 \qquad
 \begin{array}{r}
 1 0 1 \overline{) 1 1} \\
 \underline{ 1 1} \\
 0 1 1 \\
 \underline{ 1 1} \\
 0 0
 \end{array}$$

Pelo tanto, o número final obtido é o $11_{(2)}$.

6 >> Métodos para representar números enteros

Os ordenadores utilizan 4 métodos para a representación interna dos números enteiros (positivos e negativos), que son:

- Signo e magnitude (ou signo e módulo)
- Complemento a 1
- Complemento a 2
- Exceso a 2^{n-1}

Nestas representacións de números utilizamos o sistema binario e considérase que temos un número limitado de bits para cada dato numérico (a cantidade de bits soe coincidir coa lonxitude da palabra do ordenador, que soe ser de 16, 32 ou 64 bits). Este número de bits dispoñibles represéntase por n .

6.1 > Signo e magnitudine

Neste sistema de representación o bit que está situado máis á esquerda representa o **signo**; o seu valor será 0 para o signo positivo e 1 para o signo negativo. O resto de bits (n-1) representa a **magnitude** do número.

6.2 > Complemento a 1

Este sistema de representación utiliza o bit máis á esquerda para o signo, correspondendo o 0 para o signo positivo e o 1 para o signo negativo. Para os números positivos, os $n-1$ bits da dereita representan a magnitude (igual que no caso anterior). Os números negativos obtéñense complementando tódolos díxitos do positivo correspondente (cambiando 0's por 1's e viceversa), incluído o bit de signo.

6.3 > Complemento a 2

Este sistema de representación tamén utiliza o bit máis á esquerda para o signo, correspondendo o 0 para o signo positivo e o 1 para o signo negativo.

Para os números positivos, os $n-1$ bits da dereita representan a magnitude (igual que nos casos anteriores). Os números negativos obtéñense en dous pasos: primeiro calculando o complemento a 1 e segundo sumándolle 1 a ese resultado, desprezando o derradeiro acarreo se existe.

6.4 > Exceso a 2^{n-1}

Este sistema de representación non usa ningún bit para o signo, de tal xeito que todos os bits representan unha magnitude ou valor. Este valor correspóndese co número representado máis o exceso, que para n bits ven representado por 2^{n-1} .

Consiste en representar o cero como un valor intermedio, que para n bits se representa mediante 2^{n-1} e situar os números negativos antes dese valor e os positivos despois del (motivo polo que se coñece como *exceso*).

Exemplos

Representar os números 17 e -17 nunha palabra de 8 bits cos diferentes métodos descritos.

Cada número decimal ten unha representación binaria correspondente; o 17 correspóndese con 10001_2 .

Se temos 8 bits, o primeiro representa o signo e os sete restantes representan a magnitude do número:

Signo e magnitude

17	0	0 0 1 0 0 0 1
-17	1	0 0 1 0 0 0 1

Complemento a 1

17	0	0 0 1 0 0 0 1
-17	1	1 1 0 1 1 1 0

Complemento a 2

17	0	0 0 1 0 0 0 1
----	---	---------------

Para calcular o negativo, o primeiro paso é calcular o complemento a 1 do número positivo:

1	1 1 0 1 1 1 0
---	---------------

O segundo paso é sumar 1:

		+ 1
-17	1	1 1 0 1 1 1 1

Exceso a 2^{n-1}

Para 8 bits o exceso é $2^{8-1} = 2^7 = 128$. Por tanto, para representar os números pedidos teremos que lles sumar esa cantidade.

17 => 17 + 128 = 145	1 0 0 1 0 0 0 1
-17 => -17 + 128 = 111	0 1 1 0 1 1 1 1

7 >> Importancia da representación en complementos

Úsase para que dentro das máquinas non se teñan que ter máis circuítos dos necesarios. A vantaxe clara que se ten é que as restas realizarémolas como sumas, e por tanto a nosa unidade aritmético-lóxica non terá que incorporar

un restador (cun circuíto sumador bastará para realizar tanto sumas como restas).

Outra das vantaxes que teñen os complementos é que para realizar as operacións inversas fanse os mesmos pasos que para as operacións iniciais.

Se traballamos con números en **Complemento a 1** teremos que poderemos restar un número doutro simplemente sumándolle ao minuendo o complemento a 1 do subtraendo.

Ó obter o resultado, hai dúas posibilidades:

- Se hai acarreo, o resultado é un número positivo e debemos sumar ese acarreo ó resultado obtido.
- Se non hai acarreo final, o resultado é un número negativo e debemos calcular o seu complemento a 1 para obter o resultado correcto.

Exemplos

Restar o número 17 ó número 42 en palabras de 8 bits usando o complemento a 1

42	0 0 1 0 1 0 1 0
-17	0 0 0 1 0 0 0 1
Complemento a 1 de 17	1 1 1 0 1 1 1 0
Sumamos os dous números	1 0 0 0 1 1 0 0 0
Sumamos o acarreo	1
O número buscado é o 25	0 0 0 1 1 0 0 1

Restar o número 42 ó número 17 en palabras de 8 bits usando o complemento a 1

17	0 0 0 1 0 0 0 1
-42	0 0 1 0 1 0 1 0
Complemento a 1 de 42	1 1 0 1 0 1 0 1
Sumamos os dous números	0 1 1 1 0 0 1 1 0
Como o bit de acarreo é 0, complementamos o resultado	
O número buscado é o -25	1 0 0 0 1 1 0 0 1

Se estamos a traballar en **Complemento a 2** usariamos o mesmo algoritmo coa única diferenza de sumarlle 1 ó complemento a 1 para facer o complemento a 2.

7.1 > Representación en coma fixa

O seu nome vén da posición en que se supón situado o punto decimal, que será fixa. A coma fixa é usada para os números enteiros, supoñendo o punto decimal implicitamente á dereita dos bits.

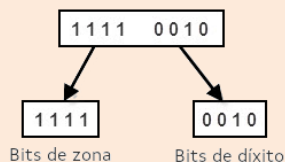
Existen catro formas de representar números en coma fixa:

- Binario puro.
- Decimal codificado en binario (BCD).
- Decimal desempaquetado.
- Decimal empaquetado.

Binario puro

Un número binario puro represéntase utilizando un conxunto de bits equivalente a unha palabra.

DECIMAL	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001



Octetos e cuartetos

Cada octeto divídese en dous cuartetos, correspondendo o da esquerda ós bits de zona e o da dereita ós bits de dígito.

Para almacenar números enteiros mediante este tipo de representación, podemos empregar os catro métodos tratados anteriormente: signo e magnitude, complemento a un, complemento a dous e exceso a 2^{n-1} , pero o habitual é empregar o **Complemento a 2**.

Exemplos

Representar os números 17 e -17 nunha palabra de 16 bits

Ó traballar cunha palabra de 16 bits, o bit máis á esquerda representa o signo (0 para positivo e 1 para negativo), mentres que os bits restantes representan a magnitude en Complemento a 2.

17 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1
 -17 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1

Decimal codificado en binario (BCD)

Para representar números mediante este sistema, cómpre converter cada dígito dun número decimal no seu equivalente de 4 bits.

Empréganse catro bits porque debemos representar 10 símbolos posibles (do 0 ó 9), e o número de símbolos representables calcúlase elevando a base (2 neste caso, ó tratarse do sistema binario) á potencia do número de bits dispoñibles: se empregásenos 3 bits, só poderíamos representar $2^3 = 8$ símbolos, polo que precisamos 4 bits para obter $2^4 = 16$ símbolos, aínda que algunhas combinacións queden sen utilizar (as combinacións concretas elixidas son as mesmas que se empregan para o código hexadecimal).

Polo tanto, simplemente precisamos substituír cada dígito dun número polo código BCD correspondente, ou viceversa.

Exemplos

Decimal a BCD

Representar o número 17 en código BCD

17 0 0 0 1 0 1 1 1

BCD a decimal

Calcular o equivalente decimal do número 0010 1001 0101 0111 almacenado en BCD

0 0 1 0 1 0 0 1 0 1 0 1 0 1 1 1 2 9 5 7

Un aspecto que cómpre ter en conta é que este sistema non permite representar números negativos; para conseguilo, sería preciso recorrer a algunha solución alternativa, como un bit de signo adicional ou algo semellante.

Para abordar esta cuestión, desenvolvéronse os seguintes métodos de representación.

Decimal sen empaquetar

A base deste sistema de representación é que cada dígito que compón un número ocupa un byte (oito bits ou un octeto).

Para codificar un dígito, cada un destes octetos divídese á súa vez en dous cuartetos (grupos de catro bits):

- O da dereita coñécese como bits de dígito e consiste na codificación BCD do dígito.
- O da esquerda coñécese como bits de zona; no octeto situado máis á dereita, estes indican o signo do número (1100 para positivo e 1101 para negativo), mentres que nos octetos restantes, os bits de zona énchense con uns.

Exemplos

Decimal sen empaquetar

Representar os números 1234 e -5678 en código decimal sen empaquetar nunha máquina con palabras de 32 bits

1234	1 1 1 1 0 0 0 1	1 1 1 1 0 0 1 0	1 1 1 1 0 0 1 1	1 1 0 0 0 1 0 0
-5678	1 1 1 1 0 1 0 1	1 1 1 1 0 1 1 0	1 1 1 1 0 1 1 1	1 1 0 1 1 0 0 0

En consecuencia, no sistema decimal sen empaquetar, a conversión é directa, pero desaproveitase unha cantidade significativa de espazo: tódolos cuartetos de zona, agás aquel que contén o signo, non conteñen información e simplemente énchense con uns.

Decimal empaquetado

Este sistema de codificación é unha evolución do anterior, deseñado para aproveitar un espazo que antes se desperdiciaba.

Cada dígito decimal represéntase mediante un cuarteto, eliminando o uso de octetos, e o signo sitúase no cuarteto máis á dereita (garantindo que ocupe sempre a mesma posición, independentemente do tamaño da palabra da máquina), seguindo a mesma codificación: 1100 para valores positivos e 1101 para os negativos.

Exemplos

Decimal empaquetado

Representar os números 1234 e -5678 en código decimal empaquetado nunha máquina con palabras de 32 bits

1234	0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 1 1 0 1 0 0 1 1 0 0
-5678	0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 1 0 0 1 1 1 1 0 0 0 1 1 0 1

7.2 > Representación en coma flotante

Ata o de agora, tratamos os números enteiros sen preocuparnos de como se almacenan os números con parte decimal.

A representación en coma flotante (ou en punto flotante no ámbito anglosaxón) permite representar números decimais e baséase na notación científica ou exponencial.

Nesta notación, un número represéntase do seguinte xeito:

$$\text{Número} = \text{mantisa} \cdot \text{base}^{\text{expoñente}}$$

Por exemplo, en base 10, a representación do número 25.4 sería:

$$0.254 \cdot 10^2$$

Para normalizar a mantisa, debemos realizar as operacións matemáticas necesarias para eliminar a parte enteira e garantir que a primeira cifra tras o punto decimal sexa significativa (é dicir, distinta de cero).

Esta operación varía segundo o número sexa menor ou maior que a unidade (en valor absoluto; o signo é irrelevante, xa que se trata por separado):

- Se o número é menor que a unidade, xa ten un punto decimal. Neste caso, comprobamos se a primeira cifra é significativa: se o é, non fai falta facer nada máis; se non, multiplicamos o número pola base do expoñente tantas veces como sexa preciso ata que a primeira cifra pase a ser significativa.

- Se o número é maior que a unidade, débémolo dividir pola base do expoñente ata obter un número que cumpra as condicións mencionadas.

Dado que agora dispoñemos de tres compoñentes (signo, expoñente e mantisa) para representar un número, debemos distribuír entre eles os bits da palabra de computador, tal como se amosa no seguinte exemplo:

31	30	24	23	0
Signo	Expoñente	Mantisa		

onde:

- **Signo:** é o código asociado ao signo; 0 para os positivos e 1 para os negativos.
- **Expoñente:** represéntase en signo e magnitude ou en exceso a 2^{n-1} , sendo sempre un número enteiro.
- **Mantisa:** é un número real con punto decimal implícito á esquerda dos seus bits, sendo representada xeralmente en signo e magnitude ou en complemento a 1 ou en complemento a 2.
- **Base do expoñente:** é unha potencia de 2 determinada polo fabricante do procesador.

Exemplos

Un ordenador utiliza o seguinte formato para representar os números en coma flotante:

- o bit 31 úsase para representar o signo da mantisa
- os bits do 23 ao 30 úsanse para representar o expoñente en exceso a 2^{n-1}
- os bits do 0 ao 22 úsanse para representar a mantisa normalizada en Complemento a 1
- a base do expoñente é 2
- o 0 representarémolo con tódolos bits a 0

(Se non estás seguro de por que é necesario especificar a representación de 0, proba un experimento rápido e calcula o número representado polos seguintes números de coma flotante: 00...00 e 11...11).

Representar os números 12 e -12 nese formato

O primeiro que teremos que facer é normalizar o número 12 a unha potencia da base, para calcular o expoñente:

$$12 = 12 \cdot 2^0 = 6 \cdot 2^1 = 3 \cdot 2^2 = 1.5 \cdot 2^3 = 0.75 \cdot 2^4$$

Polo tanto temos que o expoñente é 4.

Como a representación do expoñente é en exceso a 2^{n-1} , ó ser $n = 8$ bits teremos que $2^{8-1} = 2^7 = 128$:

$$128 + 4 = 132$$

1 0 0 0 0 1 0 0

A mantisa é 0.75 e témola que pasar a binario (multiplicando por 2), co que nos queda 0.11_2 .

Como é un número positivo quedará igual en complemento a 1. O signo deste número será positivo polo que poremos un 0.

Así 12 representarémolo como:

12 0 1 0 0 0 0 1 0 0 1 1 0

Para representar o -12 seguiremos os mesmos pasos cambiando o signo, que agora será negativo, e polo tanto un 1 e complementando a 1 a mantisa por tratarse dun número negativo:

-12 1 1 0 0 0 0 1 0 0 0 0 1

8 >> Métodos de enderezamento

Ata o de agora, aprendemos a codificar datos e a manexar sistemas que nos permiten representar números dun xeito máis complexo para nós, pero máis sinxelo para o ordenador.

Aínda precisamos entender como recuperalos unha vez almacenados; esta é a función dos métodos de enderezamento.

Ó falar de enderezamento, introduciremos un novo termo: **instrución**. Empregamos este termo para referirnos tanto a datos coma a ordes; de feito, ambos adoitan estar contidos nela, compartindo o mesmo sistema de almacenamento, e o feito de que funcionen como ordes ou como datos depende de como se utilicen.

Os métodos ou modos de enderezamento das instrucións indican como localizar o **andel** de almacenamento específico onde se garda a información: unha localización coñecida como **endereço de memoria**.

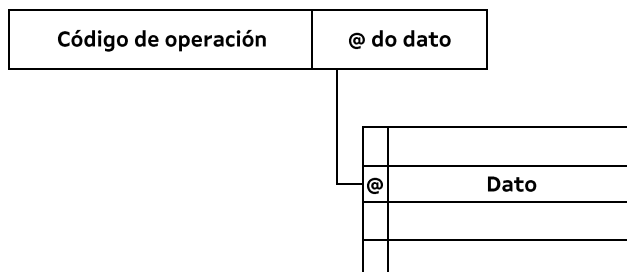
Un endereço de memoria serve como identificador dunha área de almacenamento concreta.

Poden empregarse diversos métodos de enderezamento con diferentes velocidades de execución, dependendo do número de accesos intermedios á memoria que o procesador deba realizar:

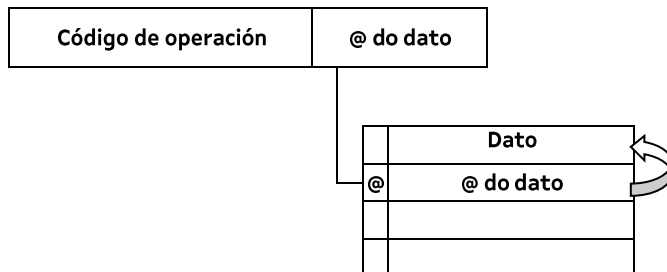
- **Enderezamento inmediato:** non precisa acceso a memoria, xa que o dato que se procesará forma parte da instrución.



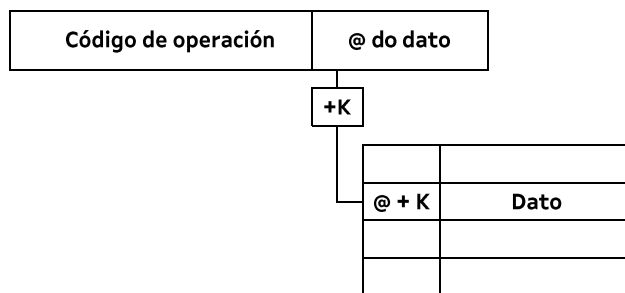
- **Enderezamento directo:** o procesador, a través da unidade de control, ten que acceder unha vez á memoria no endereço que indica a instrución para localizar o dato.



- **Enderezamento indirecto:** primeiro accedemos a unha posición de memoria que contén o endereço do dato que intervéñ na instrución.



- **Enderezamento relativo:** o endereço do dato que intervéñ na instrución obtémolo sumándolle ó endereço da propia instrución unha cantidade fixa, que normalmente está contida nun rexistro de tipo especial.



Exemplos

Buscar datos utilizando diferentes métodos de enderezamento

Dispomos dun sistema que procesa instrucións de 8 bits. Os cinco primeiros bits empréganse para especificar o código de operación, e os bits restantes utilízanse para o enderezo dos datos. Calcula os datos que se cargarán na memoria empregando os modos de enderezamento inmediato, directo, indirecto e relativo (cun desprazamento de 4) para a instrución 11011111, tendo en conta o seguinte contido da táboa de memoria do ordenador:

Enderezo	Contido
000	11001100
001	00000000
010	11111111
011	00000011
100	11110000
101	00111100
110	00000001
111	00001000

Descompoñemos a instrución indicada nas súas dúas partes:

Instrución	Código de operación	Enderezamento
1 1 0 1 1 1 1 1	1 1 0 1 1	1 1 1

Enderezamento inmediato

Como o dato xa o temos dentro da propia instrución, o único que facemos é completar con zeros a instrución:

Instrución resultante 0 0 0 0 1 1 1

Enderezamento directo

Buscamos na táboa o contido do enderezo 111:

Instrución resultante 0 0 0 1 0 0 0

Enderezamento indirecto

Buscamos na táboa o contido do enderezo 111:

Instrución intermedia 0 0 0 1 0 0 0

Volvemos buscar agora o contido do enderezo 000 (derradeiros 3 díxitos da instrución intermedia):

Instrución resultante 1 1 0 1 1 0 0

Enderezamento relativo

Sumamos o enderezo inicial (111₍₂₎) co desprazamento da máquina (4 = 100₍₂₎), desprezando o acarreo:

111 + 100 = 011

Instrución intermedia 0 0 0 0 0 1 1

Volvemos buscar agora o contido do enderezo 011 (derradeiros 3 díxitos da instrución intermedia):

Instrución resultante 0 0 0 0 0 1 1